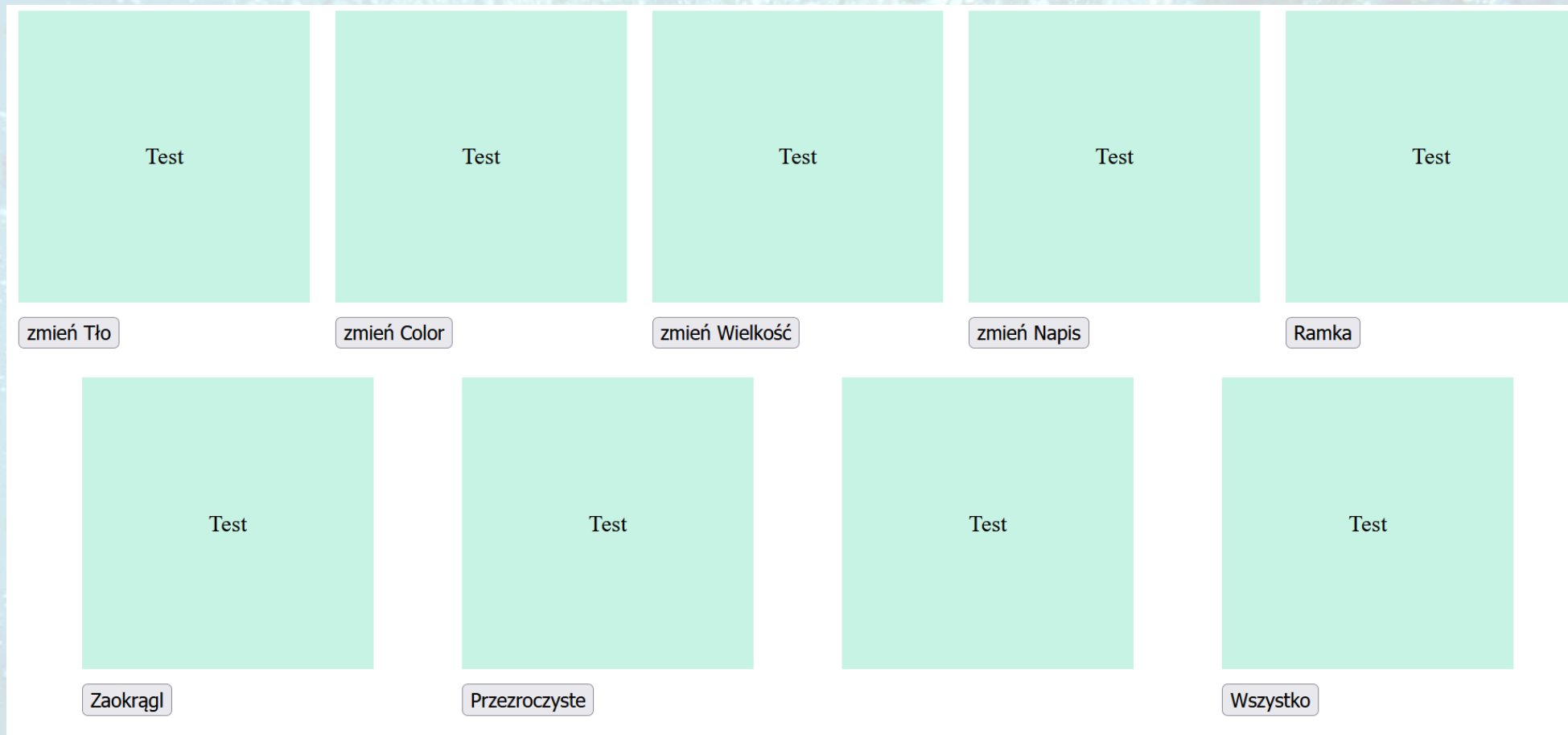


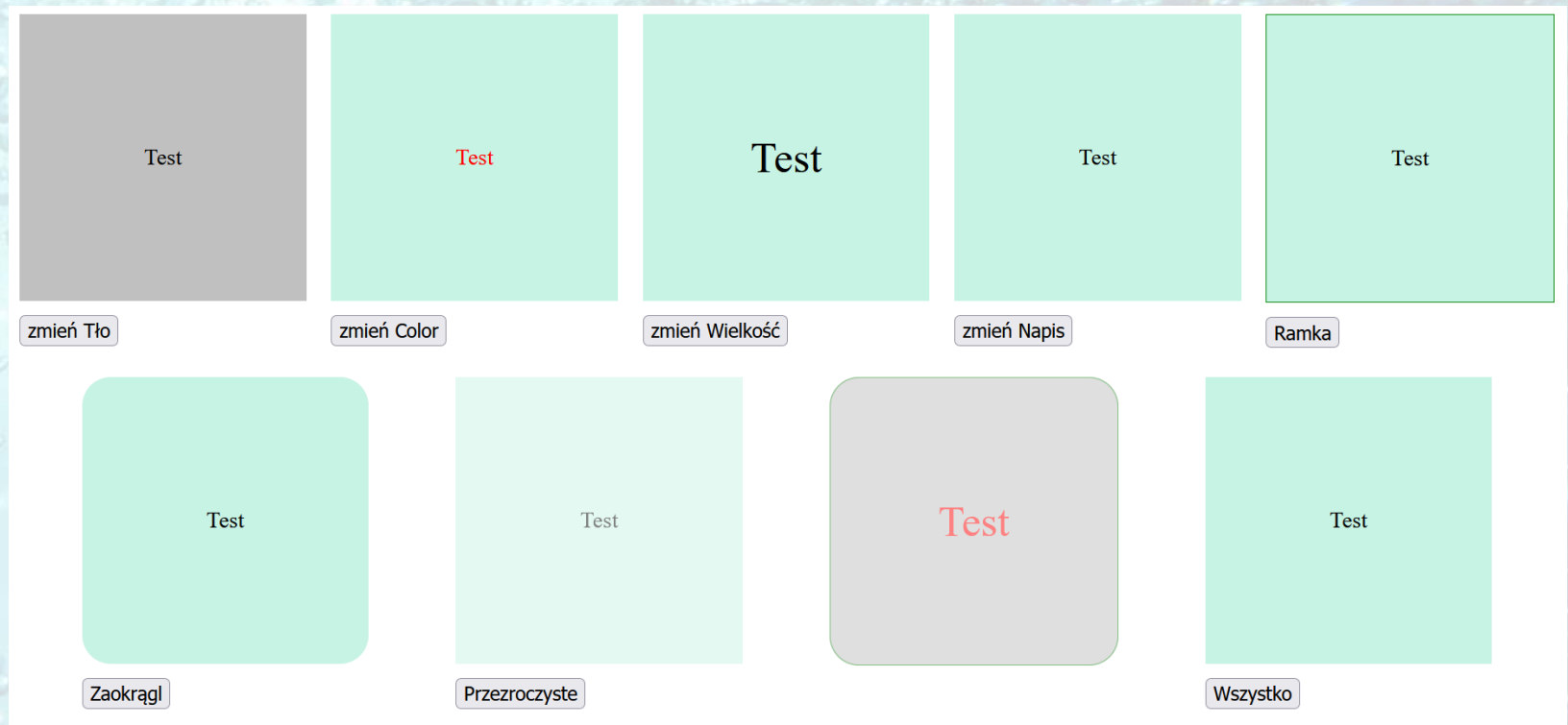
CSS - JS

Podstrona 1



Podstrona 1

Wciśnięcie przycisków powoduje wykonanie pewnego kodu w JS powodującego zmiany wyglądu (CSS).



Zmiana CSS w JS

Mając w JavaScript dostęp do elementu bierzemy jego właściwość style, a następnie właściwość CSS i następnie nadajemy jej nowe wartości. Właściwości CSS używane w JS mają nazwy jak zwykłym CSS, za wyjątkiem nazw wielowyrazowych gdzie zamiast myślników stosuje się zapis wielbłądzi.

```
function zT(){  
    document.getElementById("d1").style.backgroundColor="silver";  
    document.getElementById("d8").style.backgroundColor="silver";  
}  
function zC(){  
    document.getElementById("d2").style.color="red";  
    document.getElementById("d8").style.color="red";  
}
```


Podstrona 2

Zmiana parametrów obrazu w JS



Szarość



Jasność



Przezroczystość



Sephia



Zdarzenie *oninput* (podstrona2)

Zmiana parametrów rysunku dokonywana jest w funkcjach. Funkcji tych jednak nie wywołują zdarzenia *onclick* ale *oninput*.

```
<label for="suwak4" class="suw">Sephia</label>  
<input type="range" id="suwak4" oninput="zmianaSepi()"><br><br>
```

Klasa suw w stylizacji etykiet wykorzystuje
właściwość *display*

```
.suw{  
    width: 200px;  
    display: inline-block;  
    font-size: x-large;  
}
```


Funkcje w JS zmieniające właściwości CSS (podstrona2)

Ustawienie sepia w JS

```
document.getElementById("kot").style.filter = 'sepia(50%)';
```

Wartość liczbową podaną jako argument sepia, musi być przekazana jako string (jak powyżej). Aby zmienną zamienić na stringa można ją np. konkatelować.

```
document.getElementById("kot").style.filter = 'sepia('+odcien+')';
```

Funkcja zmianaSepi() (podstrona2)

Kod funkcji zmieniającej wartość sepii zgodnie ze ustawieniami użytkownika wybranymi suwakiem:

```
function zmianaSepi(){  
    let odcien=document.getElementById("suwak4").value;  
    odcien/=100;  
    document.getElementById("kot").style.filter = 'sepia('+odcien+')';  
}
```


Z pewnego arkusza egzaminacyjnego:

Tabela 3. Przykłady zastosowania właściwości CSS filter ze strony w3schools

Filter	Description
<code>none</code>	Default value. Specifies no effects
<code>blur(px)</code>	Applies a blur effect to the image. A larger value will create more blur.
<code>brightness(%)</code>	Adjusts the brightness of the image. 0% will make the image completely black. 100% is default and represents the original image. Values over 100% will provide brighter results.
<code>contrast(%)</code>	Adjusts the contrast of the image. 0% will make the image completely black. 100% (1) is default, and represents the original image. Values over 100% will provide results with more contrast.
<code>grayscale(%)</code>	Converts the image to grayscale. 0% (0) is default and represents the original image. 100% will make the image completely gray (used for black and white images). Note: Negative values are not allowed.
<code>invert(%)</code>	Inverts the samples in the image. 0% (0) is default and represents the original image. 100% will make the image completely inverted. Note: Negative values are not allowed.
<code>opacity(%)</code>	Sets the opacity level for the image. The opacity-level describes the transparency-level, where: 0% is completely transparent. 100% (1) is default and represents the original image (no transparency).
<code>sepia(%)</code>	Converts the image to sepia. 0% (0) is default and represents the original image. 100% will make the image completely sepia. Note: Negative values are not allowed.

Examples

Apply a blur effect to the image: `img { filter: blur(5px); }`

Adjust the brightness of the image: `img { filter: brightness(200%); }`

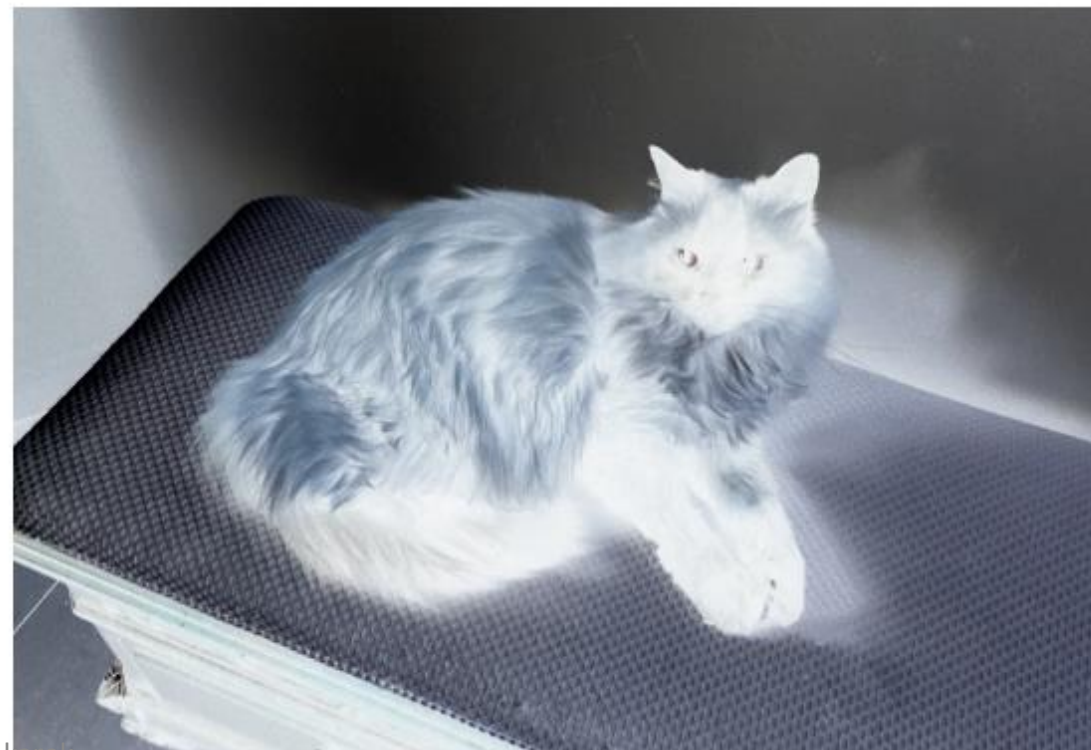
Podstrona 3

Porównanie zmian zrobionych CSS i JS

Zmiany robione CSS



Zmiany robione JS



Zmiany robione w CSS część CSS (strona lewa)

```
#kot1:hover{  
|   filter:invert(1);  
}
```

Zmiany robione JS część HTML (strona prawa)

W HTML podpinamy funkcje do dwóch zdarzeń:

```

```


Zmiany robione JS część JS (strona prawa)

W JS definiujemy funkcje dopięte do zdarzeń:

```
function inwersja1(){  
|   document.getElementById("kot2").style.filter="invert(1)";  
}  
function inwersja2(){  
|   document.getElementById("kot2").style.filter="invert(0)";  
}
```

querySelector demo (podstrona 4)

Metoda querySelector()

Metoda `querySelector()` zwraca pierwszy element pasujący do selektora CSS. Elementem może być zatem znacznik, klasa, element z identyfikatorem itp, ale zawsze tylko jeden.

Selektorem znacznik *div*:

Tekst przed zmianą

Selektorem znacznik *div*:

Po zmianie

Selektorem klasa *.szary*:

Selektorem klasa *.szary*:

Małgorzata Tokarek

Selektorem identyfikator *#gruby*:

Tekst o zmiennej grubości
czcionki

Selektorem identyfikator *#gruby*:

**Tekst o zmiennej grubości
czcionki**

querySelector – kod do demo (podstrona 4)

```
function Zmiana(){  
    document.querySelector("div").innerHTML = "Po zmianie";  
    document.querySelector(".szary").style.backgroundColor="grey";  
    document.querySelector("#gruby").style.fontWeight="bold";  
}
```

querySelectorAll demo (podstrona 5)

Metoda querySelector()

Metoda `querySelectorAll()` zwraca wszystkie elementy pasujące do selektora CSS. Elementem może być zatem znacznik, klasa, element z identyfikatorem itp. Elementy są zwracane do tablicy, a dostęp do nich mamy po indeksach.

Div 1

Div 2

Div 3

zmien

Div 1

Div 2

Div 3

querySelectorAll

W JS tworzymy zmienną, która stanie się tablicą, do której trafią wszystkie elementy mające znacznik *div*:

```
let divy;  
divy = document.querySelectorAll("div");
```

Do poszczególnych elementów tablicy (div'ów) dostajemy się po indeksie:

```
for (let i = 0; i < divy.length; i++)  
  divy[i].style.backgroundColor = "blue";
```


Cecha *position* w CSS

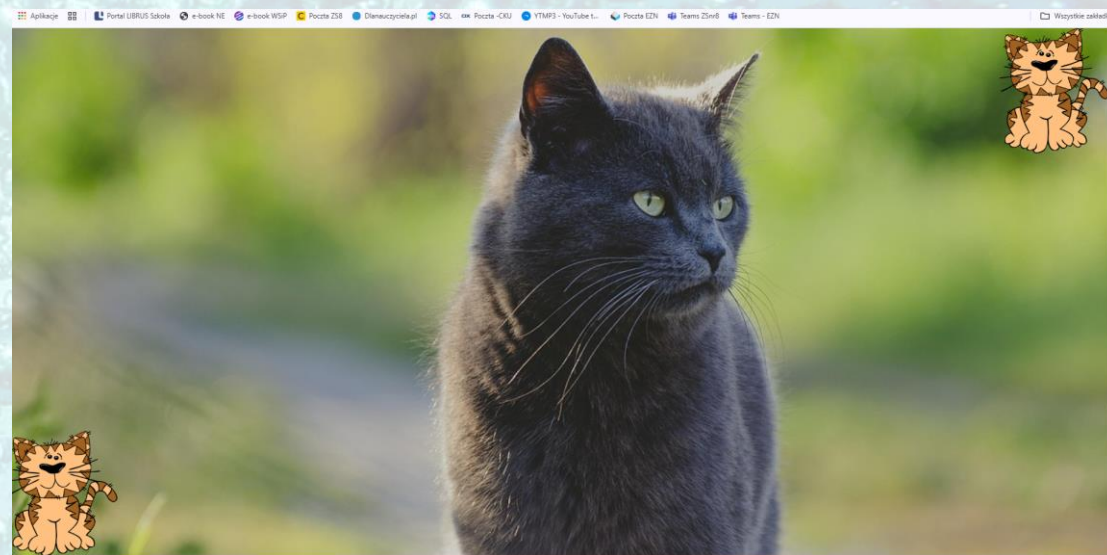
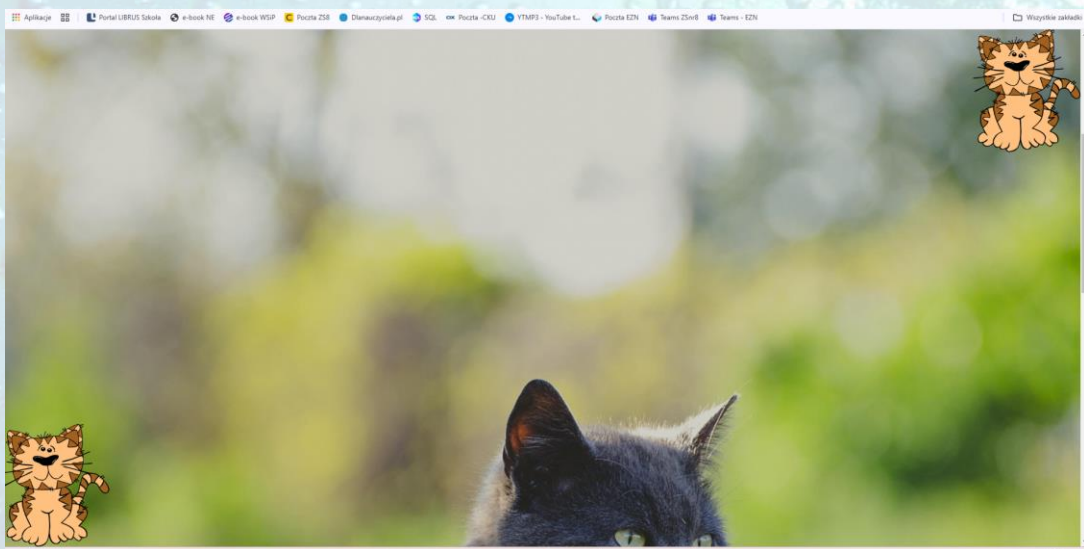
Właściwość *position* w CSS służy do określenia sposobu pozycjonowania elementu HTML na stronie internetowej. Pozwala ona na precyzyjne kontrolowanie, gdzie dany element ma się pojawić i jak ma reagować na inne elementy. Domyślną wartością jest *static*, które pozwala elementom HTML układać się w naturalnym porządku.

Położenie *position : fixed*;

Element jest wyjmowany z normalnego przepływu dokumentu i pozycjonowany względem okna przeglądarki. Oznacza to, że element pozostaje w tym samym miejscu, nawet podczas przewijania strony. Właściwości top, right, bottom i left określają odległość elementu od krawędzi okna przeglądarki.

Położenie *position : fixed*;

W przykładzie umieszczono dwa elementy z `position : fixed`; Jeden w prawym górnym rogu, drugi lewym dolnym. Podczas przesuwania strony w dół ich położenie się nie ulega zmianie.



Położenie *position : fixed*;

```
#k3{  
  width: 100%;  
  z-index: -1;  
}  
#k4{  
  width: 10%;  
  position: fixed;  
  right: 0%;  
  top:0%;  
}  
#k5{  
  width: 10%;  
  position: fixed;  
  left: 0%;  
  bottom:0%;  
}
```

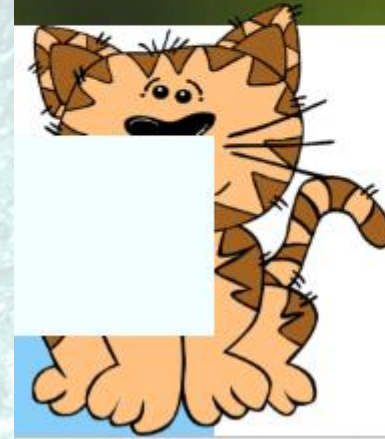
Pierwsza stylizacja dotyczy kota w tle, a cecha *z-index* służąca do określania kolejności nakładania się elementów na siebie, gdy zachodzi taka potrzeba, przesuwa go na spód. Pozostałe dwa rysunki zostają „zafiksowane” , przy czym rysunek o id=„k4” będzie się zawsze znajdował w prawym górnym rogu, a rysunek o id=„k5” w lewym dolnym.

Położenie *position : relative*

Element jest pozycjonowany względem swojego normalnego położenia w dokumencie. Można użyć właściwości *top*, *right*, *bottom* i *left*, aby przesunąć element od jego pierwotnej pozycji. Inne elementy na stronie nie zmieniają swojego położenia, aby zrobić miejsce dla przesuniętego elementu.

Położenie *position : relative*

Do poprzedniej strony dołączono dwa divy. Znalazły się one pod zdjęciem dużego kota, bo taki jest ich naturalny układ. Jednak jaśniejszy pojemnik został przesunięty w dół o 50px w stosunku do miejsca, gdzie by się znajdował.



```
#d1{  
  width: 100px;  
  height: 100px;  
  background-color: □ azure;  
  position: relative;  
  top: 50px;  
}
```


Położenie *position* : *absolute*

Element jest wyjmowany z normalnego przepływu dokumentu i pozycjonowany względem najbliższego elementu nadrzędnego, który ma ustawioną inną wartość właściwości *position* niż *static*. Jeśli żaden element nadrzędny nie ma takiego ustawienia, element jest pozycjonowany względem początku dokumentu. W tym przypadku właściwości *top*, *right*, *bottom* i *left* określają odległość elementu od krawędzi elementu nadrzędnego lub od krawędzi dokumentu.

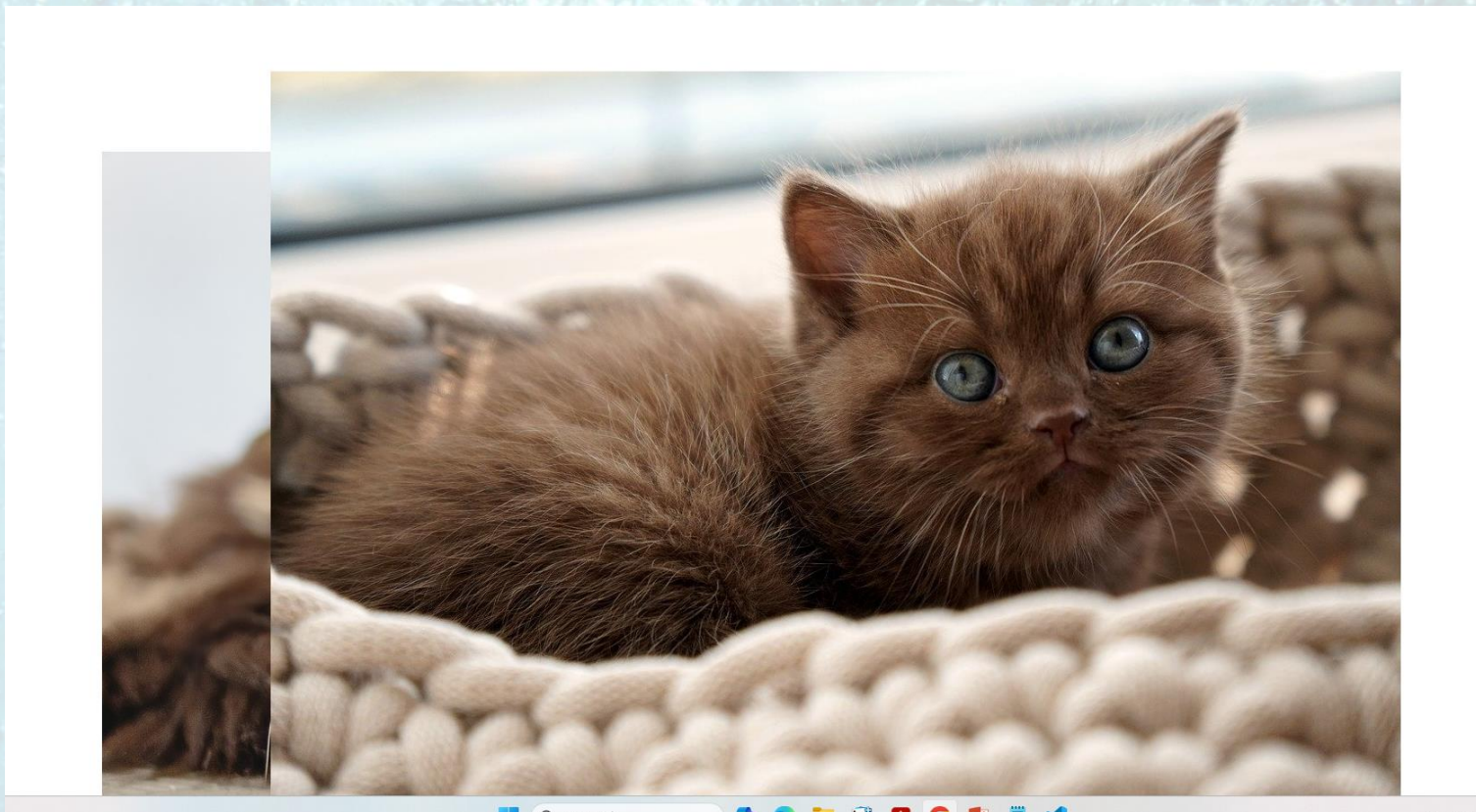
Położenie *position : absolute*

W podanym przykładzie dwa spore zdjęcia umieszczono „prawie” w tym samym miejscu. Wartości właściwości *z-index* pokazują, że zdjęcie o id="i2" znajdzie się na wierzchu (im większa wartość *z-index* tym bardziej na wierzchu znajduje się dany element).

```
#i1{
    left:10%;
    top:20%;
    z-index: 1;
    position:absolute;
}
#i2{
    left:20%;
    top:10%;
    z-index: 2;
    position:absolute;
}
```

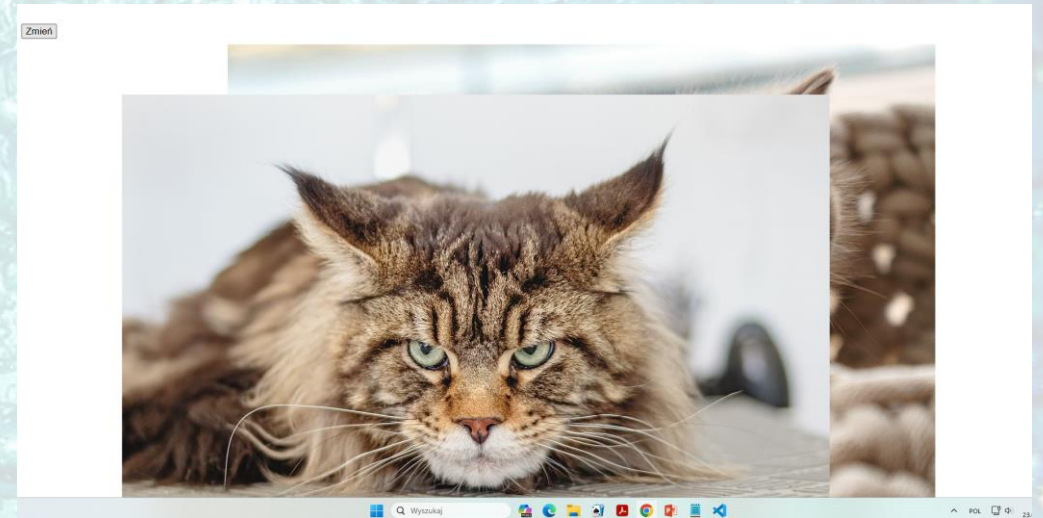

Położenie *position* : *absolute*

Rysunek przedstawia realizację stylizacji z poprzedniego slajdu:



Przykład

Zadanie do realizacji: przycisk przesuwa zdjęcie – wierzch - spód



Przykład - rozwiązanie

HTML

```
  
  
<br><br>  
<input type="button" value="Zmień" onclick="zIndexM()">
```

CSS

```
#i1{  
    left:10%;  
    top:20%;  
    z-index: 1;  
    position:absolute;  
}  
#i2{  
    left:20%;  
    top:10%;  
    z-index: 2;  
    position:absolute;  
}
```


Przykład - rozwiązanie

JS – zmienna lokalna przechowuje informację, które zdjęcie będzie na wierzchu przy kolejnej zmianie – liczba nieparzysta to na wierzch wejdzie zdjęcie 1, a przy parzystej na wierzch wejdzie zdjęcie 2

```
let licznikZmian=1;
function zIndexM(){
    if (licznikZmian%2==1){
        document.getElementById("i1").style.zIndex="2";
        document.getElementById("i2").style.zIndex="1";
        licznikZmian++;
    }
    else{
        document.getElementById("i1").style.zIndex="1";
        document.getElementById("i2").style.zIndex="2";
        licznikZmian++;
    }
}
```